APEX RETAIL GROUP | ANALYTICS TRAINING PROGRAM
DOCUMENT: CLASS NOTES (SESSION 2)
TOPIC: NAVIGATING BUSINESS RULES — LOGIC, CONDITIONS & DATA VALIDATION
LEVEL: BEGINNER (DAY 2)

1. CONTINUITY & BUSINESS SCENARIO RECAP

In Session 1, you calculated basic team-level metrics (**sum**, **mean**, **individual variances**)for an HR salary review. You accessed values using manual index positioning (**e.g., `salaries[0]`**).

Now, it is your second week at Apex Retail Group.

Marcus Vance, the Operations Lead for E-Commerce Fulfillment, approaches you with a more dynamic problem:

> "Our regional distribution centers process thousands of online orders daily. Our customer promise is a strict **Service Level Agreement (SLA)**: *Standard orders must ship within 24 hours, while Express orders must ship within 12 hours.*
>
> Customers are complaining about fulfillment delays, but we don't know which orders are breaching our SLA terms. Manual checking is impossible.
>
> I have a sample batch of order processing times in hours, along with their order types. **Can you write a programmatic evaluation that automatically flags which orders violated our SLA terms, calculates our overall SLA compliance rate, and handles any bad data entries?"**

In this session, you will learn how to teach a computer to make decisions automatically using conditional logic, iterate through batches of data using loops, encapsulate logic into reusable functions, and gracefully handle corrupted data values.

2. TRANSLATING BUSINESS RULES INTO LOGICAL CONDITIONS

A business rule is a conditional statement. Before writing code, an analyst maps out the decision tree in plain language:

Business SLA Rule	Translated Logical Condition
Express Order Violation	**IF** order_type == "Express" **AND** hours > 12.0
Standard Order Violation	**IF** order_type == "Standard" **AND** hours > 24.0
Compliant Order	OTHERWISE (**ELSE**) --> Order is SLA Compliant

3. PROGRAMMATIC BUILDING BLOCKS FOR AUTOMATED ANALYSIS

A. Comparison Operators

To evaluate conditions, Python uses comparison operators that return Booleans (`True` or `False`):

- `==` Equal to
- `!=` Not equal to
- `>` Greater than
- `<` Less than
- `>=` Greater than or equal to
- `<=` Less than or equal to

B. Conditional Branching (`if`, `elif`, `else`)

Conditional statements allow code to execute specific actions based on whether a condition is true:

```
if processing_hours > 24.0:
    status = "SLA Breach"
else:
    status = "Compliant"
```

C. Automated Iteration: `for` Loops

In Session 1, we accessed elements individually (`list[0]`, `list[1]`). When processing dozens or thousands of order records, manual indexing fails.

A `for` loop automatically iterates through every item in a collection sequentially:

```
for hours in order_processing_times:
   # Process each order hour value automatically
```

D. Reusable Business Logic: Functions (`def`)

Instead of rewriting business logic rules repeatedly, analysts encapsulate rules into FUNCTIONS. A function takes input parameters, executes business logic, and returns a result:

```
def check_sla_status(order_type, processing_hours):
    if order_type == "Express" and processing_hours > 12.0:
        return "Express Breach"
    elif order_type == "Standard" and processing_hours > 24.0:
        return "Standard Breach"
    else:
        return "Compliant"
```

E. Handling Corrupted Data: Basic Error Handling (`try` / `except`)

Real-world business data is rarely clean. A dataset might contain non-numeric corrupt values (e.g., `"INVALID"`, `"UNKNOWN"`, or missing entries).

Without error handling, a non-numeric string crashes the entire analysis script. We use `try` / `except` blocks to catch errors, flag bad rows, and keep processing:

```
try:
    hours = float(raw_value)
except ValueError:
    hours = None  # Flag as bad/missing data rather than crashing
```

4. EXECUTING THE FULFILLMENT AUDIT IN PYTHON

Below is the clean, annotated Python code required to perform Marcus's fulfillment SLA audit:

```python
# Step 1: Define SLA Benchmark Constants
EXPRESS_SLA_LIMIT = 12.0
STANDARD_SLA_LIMIT = 24.0

# Step 2: Raw batch order dataset provided by Operations
# Contains order IDs, order types, and logged processing times (with corrupted data)
order_batch = [
    {"order_id": "ORD-101", "type": "Standard", "hours": 18.5},
    {"order_id": "ORD-102", "type": "Express",  "hours": 14.0},
    {"order_id": "ORD-103", "type": "Standard", "hours": 29.0},
    {"order_id": "ORD-104", "type": "Express",  "hours": 8.5},
    {"order_id": "ORD-105", "type": "Standard", "hours": "CORRUPTED_DATA"},
# Data quality issue
    {"order_id": "ORD-106", "type": "Express",  "hours": 11.5},
    {"order_id": "ORD-107", "type": "Standard", "hours": 26.5}
]

# Step 3: Define SLA Evaluation Function with Data Validation
def evaluate_order_sla(order_type, raw_hours):
    # Error handling for bad data inputs
    try:
        hours = float(raw_hours)
    except (ValueError, TypeError):
        return "Data Error"

    # Business logic evaluation
    if order_type == "Express" and hours > EXPRESS_SLA_LIMIT:
        return "SLA Breach"
    elif order_type == "Standard" and hours > STANDARD_SLA_LIMIT:
        return "SLA Breach"
```

```
    else:
        return "Compliant"

# Step 4: Process the order batch using a loop
breach_count = 0
compliant_count = 0
error_count = 0

print("--- INDIVIDUAL ORDER AUDIT LOG ---")
for order in order_batch:
    status = evaluate_order_sla(order["type"], order["hours"])
    print(f"Order {order['order_id']} ({order['type']}): {status}")

    if status == "SLA Breach":
        breach_count += 1
    elif status == "Compliant":
        compliant_count += 1
    elif status == "Data Error":
        error_count += 1

# Step 5: Compute Compliance Rate Metrics
valid_orders_count = breach_count + compliant_count
compliance_rate = (compliant_count / valid_orders_count) * 100

print("\n--- OPERATIONS FULFILLMENT SUMMARY ---")
print(f"Total Orders Evaluated: {len(order_batch)}")
print(f"Valid Orders Processed: {valid_orders_count}")
print(f"Compliant Orders:       {compliant_count}")
print(f"SLA Breaches:           {breach_count}")
print(f"Corrupted Data Logs:    {error_count}")
print(f"Fulfillment Compliance: {compliance_rate:.2f}%")
```

INTERPRETING NUMBERS & FORMULATING BUSINESS RECOMMENDATIONS

A. Observation (The Raw Data):

- Total Orders Evaluated: 7
- Corrupted Logged Entries: 1 (ORD-105 marked "CORRUPTED_DATA")
- Valid Orders Processed: 6
- Compliant Orders: 4 (ORD-101, ORD-104, ORD-106, ORD-107... wait, check ORD-107: Standard at 26.5h is a breach!)
- Correct Breakdowns:
    - ORD-102 (Express, 14.0h) -> Breach (> 12h)
    - ORD-103 (Standard, 29.0h) -> Breach (> 24h)
    - ORD-107 (Standard, 26.5h) -> Breach (> 24h)
    - Total Breaches: 3 | Total Compliant: 3
- Operational SLA Compliance Rate: 50.00% across valid orders.

B. Interpretation & Business Meaning:

- Half (50.00%) of all processed fulfillment requests are breaching established customer SLAs.
- Breaches are widespread across BOTH fulfillment lines (1 out of 3 Express orders breached; 2 out of 3 valid Standard orders breached).
- Data collection systems are dropping logging quality (14.3% data corruption rate in this sample batch), which hides operational bottlenecks.

C. Professional Recommendation to Marcus Vance (Operations Lead):

"An automated audit of the sample fulfillment batch reveals a critical 50.0% SLA breach rate across valid orders, split across both Express (>12h) and Standard (>24h) fulfillment channels. Furthermore, 14.3% of logged records contained corrupted timestamp data, indicating logging infrastructure failures.

Recommendation: Immediately pause automated SLA compliance scoring until distribution center timestamp logging is sanitized. Conduct a root-cause investigation into night-shift packing queues where processing delays spike above 26+ hours."

CORE MINDSET HABITS REINFORCED TODAY

1. Never blindly trust raw data: Always wrap data parsing in validation checks (`try`/`except`) to prevent corrupted entries from blowing up your analytical scripts.
2. Automate repetitive logic: Use functions and loops to ensure rules are applied consistently across 7 or 7,000 orders.